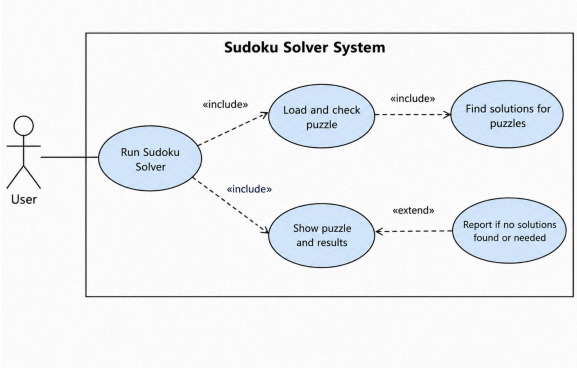
UML Use Case Model: Sudoku Solver

Assignment 4 Saylor Statler October 14, 2026

Use Case Diagram
The diagram shows how a user interacts with the Sudoku Solver System. The main use case is to run the solver, its tasks, validate a puzzle, search for solutions, and display results. Reporting that no solution exists is an optional extension when the search finds none.

Sudoku Solver System:
—----------
User - Run Sudoku Solver

<<include>>\ \<<include>>
Load and check find solutions for
puzzle puzzles
<<include>>\ \<<extend>>
Show puzzle
And results report if no solutions is
Found or needed
AI put it into proper uml:



—----------
Identify / Discover the Use Case
Primary actor:
User (the person running the program).
System:
Sudoku Solver System, C++ program.
Use case:
Run Sudoku Solver.
Goal:
Read each Sudoku puzzle file, find every valid solution, and show the original puzzle and results.

Brief Description

The user runs the program. The program opens the five puzzle files (puzzle1.txt through puzzle5.txt), checks that each puzzle is readable, correct, and searches for all valid completed sudoku boards. It prints each original puzzle, any solutions found, and the number of solutions, or that no solution was found.

Step-by-Step Flow of Events

1: The user starts the sudoku solver executable.
2: The program begins processing the five puzzle files in order.
3: For each file, the program attempts to open and read the sudoku grid.
4: The program displays the original puzzle and checks the input format and any given numbers.
5: If the puzzle is good the program searches for solutions using recursive backtracking, trying numbers in blank cells, and undoing choices that don't work.
6: Whenever a complete board is found the program displays it and continues searching for additional solutions.
7: The program displays the total number of solutions or that there is no solution found. If a file cannot be read or its format is invalid, it reports the input problem.
8: The program repeats the process for the remaining puzzle files and then finishes.

Provide a detailed description for the use case

When the user runs the sudoku solver program the system attempts to open and read each of the five puzzle files. It displays the original puzzle and checks that the input is valid before searching for solutions. If the puzzle is valid the program uses recursive backtracking to fill in the empty cells and find every possible correct solution. Each solution is displayed as a completed sudoku board and the program prints the total number of solutions found. If no solution exists the program prints that no solutions were found. If a file cannot be opened or a puzzle contains an invalid input, the program reports the problem. The system repeats this for all five puzzle files and finishes after processing them.